

Semana 4: Pensamiento algorítmico — Recetas para resolver problemas

Guía completa de la sesión

Visión general

Duración total	3 horas (2 h cátedra + 1 h laboratorio analógico)
Objetivo central	Que los estudiantes comprendan qué es un algoritmo, puedan expresar procedimientos como pseudocódigo o diagramas de flujo, e intuyan que los problemas tienen diferentes niveles de dificultad computacional
Idea ancla	Un algoritmo es una receta tan precisa que cualquier ejecutante — humano, máquina, o extraterrestre — obtiene el mismo resultado. La pregunta no es solo “¿puedo resolverlo?” sino “¿qué tan caro es resolverlo?”
Prerrequisito	Semana 1 (instrucciones precisas, el Computador Humano), Semana 3 (búsqueda binaria en las 20 Preguntas)
Materiales	Proyector, pizarra, tarjetas numeradas grandes (1–8, un set por equipo), tiza o cinta adhesiva para marcar grilla en el suelo, cronómetro, copias del Control 2
Conexión con semanas previas	Semana 1: “las instrucciones deben ser precisas” → hoy formalizamos eso. Semana 3: la búsqueda binaria en 20 Preguntas reaparece como un algoritmo con complejidad logarítmica

PARTE 1: CÁTEDRA (120 minutos)

Bloque A — Apertura: la receta de cocina (15 min)

Actividad: Pedir a un voluntario que explique, paso a paso, cómo hacer un sándwich. El docente ejecuta las instrucciones *literalmente* (como un computador).

Instrucciones típicas del estudiante y reacciones del docente: - “Pon el pan sobre la mesa” → ¿Qué pan? ¿Cuál mesa? ¿Con qué lado hacia arriba? - “Ponle jamón” → ¿Cuántas láminas? ¿En qué posición? ¿Sobre cuál cara del pan? - “Cierra el sándwich” → *Pone otro pan al lado en vez de encima.*

El curso se ríe, pero la lección es seria: las instrucciones en lenguaje natural son **ambiguas**. Un computador (y el docente haciendo de computador) las ejecuta mal.

Transición: “¿Cómo se escribe una receta que no admita ambigüedad? Eso es un algoritmo.”

Bloque B — ¿Qué es un algoritmo? (25 min)

Definición (5 min) Un **algoritmo** es un conjunto finito de instrucciones bien definidas que transforma una entrada en una salida deseada.

No es un concepto moderno: la palabra viene de **al-Juarismi** (Muhammad ibn Musa al-Khwarizmi, ~780–850), matemático persa cuyo libro describía procedimientos paso a paso para resolver ecuaciones.

Las cinco propiedades (20 min)

Propiedad	Significado	Contraejemplo (NO es algoritmo)
Finitud	Termina después de un número finito de pasos	“Repita hasta que el resultado sea perfecto” (¿cuándo es “perfecto”?)

Propiedad	Significado	Contraejemplo (NO es algoritmo)
Definición	Cada paso es preciso y no ambiguo	“Agregue sal al gusto”
Entrada	Recibe cero o más datos iniciales	(Siempre se cumple si se define bien)
Salida	Produce al menos un resultado	Un programa que nunca entrega nada
Efectividad	Cada paso es realizable en un tiempo finito con los recursos disponibles	“Calcule todos los números primos” (infinitos)

Pregunta al curso: “La instrucción ‘OPERAR celda 0 ÷ celda 1 → celda 2’ de la Semana 1, ¿cumple las cinco propiedades?” → Sí: es finita (un paso), definida (operación y celdas especificadas), tiene entrada (dos celdas), salida (una celda), y es efectiva (la ALU puede hacerlo).

Pregunta trampa: “‘Encontrar el mejor lugar para un área protegida’ — ¿es un algoritmo?” → No tal como está formulado. “Mejor” no está definido. ¿Mejor para qué especie? ¿Con qué restricción de presupuesto? ¿Minimizando qué función? La vaguedad lo descalifica. Pero si definimos “mejor” con criterios precisos (e.g., maximizar el número de especies protegidas con un presupuesto de X), entonces sí se puede construir un algoritmo.

Bloque C — Herramientas de expresión: diagramas de flujo y pseudocódigo (30 min)

Diagramas de flujo (15 min) Símbolos estándar:

Símbolo	Forma	Significado
Inicio / Fin	Óvalo	Marca el comienzo o final
Proceso	Rectángulo	Una acción o cálculo
Decisión	Rombo	Pregunta sí/no (bifurcación)
Entrada / Salida	Paralelogramo	Lectura de datos o escritura de resultado
Flecha	Línea con punta	Dirección del flujo

Ejemplo en la pizarra: Diagrama de flujo para decidir si una especie está amenazada.

```

[INICIO]
↓
[Leer: población, tendencia, rango]
↓
¿Población < 1000?
Sí → ¿Tendencia decreciente?
    Sí → [Clasificar: EN PELIGRO CRÍTICO]
    No → [Clasificar: EN PELIGRO]
No → ¿Población < 10000?
    Sí → [Clasificar: VULNERABLE]
    No → [Clasificar: PREOCUPACIÓN MENOR]
↓
[Escribir: clasificación]
↓
[FIN]

```

Dibujar paso a paso en la pizarra con los símbolos correctos. Pedir a los estudiantes que identifiquen cada símbolo.

Pseudocódigo (15 min) El pseudocódigo es un **intermedio** entre el lenguaje natural y un lenguaje de programación. No tiene una sintaxis formal — lo importante es la claridad.

Convenciones que usaremos:

LEER variable

ESCRIBIR variable

```

SI condición ENTONCES
    instrucciones
SINO
    instrucciones
FIN SI
MIENTRAS condición HACER
    instrucciones
FIN MIENTRAS
PARA i DESDE a HASTA b HACER
    instrucciones
FIN PARA

```

El mismo ejemplo en pseudocódigo:

```

LEER población, tendencia, rango
SI población < 1000 ENTONCES
    SI tendencia = "decreciente" ENTONCES
        clasificación ← "En peligro crítico"
    SINO
        clasificación ← "En peligro"
    FIN SI
SINO SI población < 10000 ENTONCES
    clasificación ← "Vulnerable"
SINO
    clasificación ← "Preocupación menor"
FIN SI
ESCRIBIR clasificación

```

Comparación: el diagrama de flujo es visual (bueno para comunicar, malo para procedimientos largos). El pseudocódigo es textual (bueno para detallar, más cercano al código real).

Ejercicio rápido (5 min): “Escriban en pseudocódigo el procedimiento para decidir si hoy necesitan abrigo al salir de casa.” Pedir a 2–3 estudiantes que lean los suyos. Discutir ambigüedades.

Bloque D — Algoritmos de ordenamiento: el caso de estudio (30 min)

¿Por qué ordenar? (5 min) “¿Por qué querríamos ordenar una lista?”

Ejemplos ecológicos: - Ordenar especies por abundancia para identificar las dominantes. - Ordenar parcelas por riqueza para priorizar las más diversas. - Ordenar datos cronológicamente para analizar tendencias.

Ordenar es una de las operaciones más comunes en computación. Existen docenas de algoritmos para hacerlo. Hoy vemos dos de los más simples.

Ordenamiento burbuja — Bubble Sort (10 min) **Idea:** recorrer la lista comparando pares adyacentes. Si están desordenados, intercambiarlos. Repetir hasta que no haya más intercambios.

Demostración con el curso: Pedir a 6 voluntarios que se pongan de pie frente al curso, cada uno sosteniendo una tarjeta con un número (por ejemplo: 5, 2, 8, 1, 9, 3).

Procedimiento: 1. Comparar posiciones 1 y 2. Si están desordenados, intercambiar. 2. Comparar posiciones 2 y 3. Mismo criterio. 3. ... hasta el final de la fila. 4. Volver al inicio. Repetir hasta que una pasada completa no tenga intercambios.

Observación: después de la primera pasada, el número más grande “burbujea” al final. Después de la segunda, el segundo más grande queda en su lugar. Etc.

Pseudocódigo:

```

PARA i DESDE 1 HASTA n-1 HACER
    PARA j DESDE 1 HASTA n-i HACER
        SI lista[j] > lista[j+1] ENTONCES

```

```

        intercambiar(lista[j], lista[j+1])
    FIN SI
FIN PARA
FIN PARA

```

Ordenamiento por inserción — Insertion Sort (10 min) **Idea:** mantener una parte “ya ordenada” de la lista e ir insertando cada nuevo elemento en su posición correcta.

Analogía: así ordenamos las cartas cuando jugamos a las cartas — tomamos una del mazo y la deslizamos al lugar correcto en la mano.

Demostración: los mismos 6 voluntarios. El primero se queda. El segundo se compara y se coloca antes o después. El tercero busca su posición entre los dos ya ordenados. Etc.

Pseudocódigo:

```

PARA i DESDE 2 HASTA n HACER
    clave ← lista[i]
    j ← i - 1
    MIENTRAS j > 0 Y lista[j] > clave HACER
        lista[j+1] ← lista[j]
        j ← j - 1
    FIN MIENTRAS
    lista[j+1] ← clave
FIN PARA

```

Comparación (5 min)

	Burbuja	Inserción
Idea	Comparar adyacentes, intercambiar	Insertar en posición correcta
Mejor caso	Lista ya ordenada: pocas pasadas	Lista ya ordenada: una pasada
Peor caso	Lista al revés: máximos intercambios	Lista al revés: máximos desplazamientos
Fácil de entender	Muy fácil	Fácil
Eficiente	No (lento para listas grandes)	Algo mejor, pero tampoco escala

“Ambos funcionan. Pero ¿qué pasa si la lista tiene un millón de elementos?”

Bloque E — Complejidad: ¿qué tan difícil es este problema? (20 min)

La pregunta (5 min) No basta con saber que un algoritmo funciona. Necesitamos saber **cuánto cuesta**.

“Costo” = **cuántas operaciones** necesita en función del tamaño de la entrada (n).

Notación Big-O (conceptual, sin formalismo) (10 min)

Complejidad	Nombre	Ejemplo	Crece como...
$O(1)$	Constante	Leer el primer elemento de una lista	Instantáneo
$O(\log n)$	Logarítmica	Búsqueda binaria (¡Semana 3!)	Muy lento
$O(n)$	Lineal	Buscar un nombre recorriendo toda la lista	Proporcional
$O(n \log n)$	Log-lineal	Algoritmos de ordenamiento eficientes (merge sort)	Razonable

Complejidad	Nombre	Ejemplo	Crece como...
$O(n^2)$	Cuadrática	Burbuja, inserción (peor caso)	Problemático
$O(2^n)$	Exponencial	Probar todas las combinaciones posibles	Intratable

Ejemplo concreto: ordenar una lista de n especies.

n (especies)	Burbuja $O(n^2)$	Merge sort $O(n \log n)$
10	~100 operaciones	~33
100	~10.000	~664
1.000	~1.000.000	~9.966
1.000.000	~1.000.000.000.000	~19.931.568

“Con un millón de especies, burbuja necesita un billón de operaciones. Merge sort necesita 20 millones. La diferencia no es ‘un poco más rápido’ — es la diferencia entre tardar segundos y tardar días.”

La búsqueda binaria revisitada (5 min) Conectar con Semana 3:

En las 20 Preguntas, la búsqueda binaria con 16 animales tardó **4 preguntas** ($\log(16) = 4$).

Si hubieran preguntado “¿es el pudú?”, “¿es el cóndor?” (búsqueda lineal): hasta **16 preguntas** en el peor caso.

$O(\log n)$ vs. $O(n)$ — la diferencia entre preguntar 20 veces y preguntar 4 veces. Y para 1.000.000 de animales: 1.000.000 preguntas vs. 20 preguntas.

Bloque F — Cierre y puente al laboratorio (10 min)

Recapitulación: 1. Un **algoritmo** es un procedimiento finito, definido, efectivo, con entrada y salida. 2. Se puede expresar como **diagrama de flujo** (visual) o **pseudocódigo** (textual). 3. Los algoritmos de **ordenamiento** son un caso de estudio clásico: misma tarea, diferentes estrategias, costos muy distintos. 4. La **complejidad** mide cuánto cuesta un algoritmo cuando crece la entrada. La diferencia entre $O(n^2)$ y $O(n \log n)$ es la diferencia entre lo posible y lo imposible.

Anticipación del laboratorio: *“Ahora van a ser los datos. Literalmente. Van a pararse en una grilla y ordenarse siguiendo dos algoritmos diferentes. Van a cronometrar cuánto tarda cada uno. Y van a descubrir en carne propia por qué el algoritmo importa.”*

PARTE 2: LABORATORIO ANALÓGICO (60 minutos)

“Red de Ordenamiento”

Preparación previa (docente)

Materiales

Material	Cantidad	Notas
Tarjetas numeradas grandes (visible a 5 metros)	2 sets de 8 tarjetas (números 1–8)	Cartulina A4 con número grande en marcador grueso
Cinta adhesiva o tiza	Suficiente para marcar 8 posiciones en el suelo	Separadas ~80 cm entre sí, en línea recta
Cronómetro	1 (celular del docente)	

Material	Cantidad	Notas
Pizarra o papelógrafo	1	Para anotar tiempos
Hoja de registro del laboratorio	6 copias (1 por grupo)	

Preparación del espacio En el auditorio de bancas fijas, usar el **pasillo central** o el **espacio frente a la pizarra** para marcar las posiciones. Si no hay espacio suficiente para 8 en línea, usar una línea de 6.

Marcar en el suelo con cinta adhesiva:

[1] [2] [3] [4] [5] [6] [7] [8]
 ↑ ↑ ↑ ↑ ↑ ↑ ↑ ↑
 posiciones fijas en el suelo

Cada posición debe ser suficientemente grande para que una persona se pare ahí.

Desarrollo de la actividad

Fase 1 — Demostración: Bubble Sort físico (15 min) **Preparación:** - 8 voluntarios se paran en las posiciones, cada uno sosteniendo una tarjeta con un número aleatorio (e.g., 5, 2, 8, 1, 9, 3, 7, 4). - Los demás estudiantes observan y cuentan los intercambios.

Procedimiento: 1. El docente (o un estudiante “director”) recorre la fila de izquierda a derecha. 2. En cada par adyacente: si el de la izquierda es mayor que el de la derecha, **se intercambian de posición** (caminan y se cruzan). 3. Al llegar al final, volver al inicio y repetir. 4. Continuar hasta una pasada sin intercambios.

Registrar: - Número total de **comparaciones** (cada vez que se mira un par). - Número total de **intercambios** (cada vez que dos personas se cruzan). - **Tiempo total** (cronómetro).

Después de la demostración: preguntar a los observadores: “¿Vieron algún patrón? ¿Qué pasó con el número más grande en la primera pasada?” → Burbujeó al final.

Fase 2 — Demostración: Insertion Sort físico (15 min) **Mismos 8 voluntarios**, reordenados aleatoriamente.

Procedimiento: 1. El primer estudiante se queda en su posición (la “parte ordenada” tiene 1 elemento). 2. El segundo se compara con el primero. Si es menor, se inserta antes (los demás se desplazan). 3. El tercero busca su posición correcta entre los ya ordenados, se inserta, los demás se desplazan. 4. Continuar hasta que todos estén en la “parte ordenada”.

Registrar: mismas métricas.

Fase 3 — Competencia por equipos (20 min) **Dividir el curso en 3 equipos de ~10 personas** (de cada equipo, 8 participan, 2 registran datos).

Cada equipo ordena la misma secuencia aleatoria (dada por el docente) usando: - **Equipo A:** Bubble Sort - **Equipo B:** Insertion Sort - **Equipo C:** “Estrategia libre” — el equipo inventa su propio método

Reglas de la competencia: - Los participantes **solo pueden comparar tarjetas con su vecino inmediato** (no vale mirar toda la fila y decidir a dónde ir directamente — eso sería un algoritmo diferente y más potente). - Los registradores cuentan comparaciones e intercambios. - El docente cronometra.

Secuencia a ordenar (misma para los tres): 7, 3, 5, 1, 8, 2, 6, 4

Después de la competencia: anotar en la pizarra:

Equipo	Algoritmo	Comparaciones	Intercambios	Tiempo
A	Burbuja			
B	Inserción			
C	Libre			

Discutir: ¿quién fue más rápido? ¿Por qué? ¿Qué hizo el equipo C? ¿Se acerca a algún algoritmo conocido?

Fase 4 — Discusión plenaria (10 min) Preguntas guía:

1. “¿Cuántos intercambios hicieron? ¿Coincide con lo que esperaríamos teóricamente?” → Para 8 elementos, burbuja peor caso: 28 comparaciones ($n(n-1)/2$), hasta 28 intercambios.
2. “¿Qué pasaría si en vez de 8 personas fueran 80? ¿O 800?” → Burbuja: $80^2 = 6.400$ comparaciones. $800^2 = 640.000$. Crece cuadráticamente.
3. “¿La estrategia libre fue más rápida? ¿Qué hicieron diferente?” → Muchos equipos intuitivamente inventan algo parecido a selection sort (buscar el mínimo, ponerlo al inicio) o a merge sort (dividir en mitades). Discutir.
4. “¿Ven alguna conexión con la búsqueda binaria de la Semana 3?” → La búsqueda binaria es $O(\log n)$. Si pudiéramos dividir la lista recursivamente y ordenar cada mitad... eso es merge sort, $O(n \log n)$. No lo vemos en detalle, pero la idea es que “dividir para conquistar” es una estrategia algorítmica poderosa.
5. “¿Qué paralelo ven con la conservación?” → Priorizar parcelas, seleccionar sitios de monitoreo, decidir dónde invertir recursos limitados — todo requiere algún tipo de ordenamiento o ranking. Y el costo del procedimiento importa cuando los datos son grandes.

PARTE 3: CONTROL 2 (15 minutos, individual, con calificación)

Aplicar al final de la sesión. No se permite uso de celular ni apuntes.

Control 2 — Semana 4

Introducción al Análisis de Datos y Programación Nombre: _____

Fecha: _____ Puntaje: _____ / 20

Tiempo: 15 minutos. Sin apuntes ni celular.

-
1. (4 puntos) Enumere las cinco propiedades que debe cumplir un algoritmo. Para cada una, escriba una frase que la explique.

[espacio]

-
2. (6 puntos) Usted debe decidir qué cuadrantes muestrear en un transecto de 20 cuadrantes. La regla es: muestrear todo cuadrante cuya cobertura vegetal sea mayor al 60%. Los datos de cobertura están en una lista.

Escriba el **pseudocódigo** de un algoritmo que recorra la lista, evalúe cada cuadrante, y produzca como salida la lista de cuadrantes seleccionados.

[espacio]

-
3. (4 puntos) Dibuje un **diagrama de flujo** para el siguiente procedimiento: “Si la temperatura es menor a 5°C , registrar ‘helada’. Si está entre 5°C y 15°C , registrar ‘frío’. Si es mayor a 15°C , registrar ‘templado’.”

[espacio]

-
4. (4 puntos) Tiene una lista de 1.000 especies. ¿Cuántas comparaciones necesita *en el peor caso* cada algoritmo para ordenarla?

a) Bubble sort: _____ (aprox.)

b) Un algoritmo $O(n \log n)$ como merge sort: _____ (aprox.)

Pista: $\log(1000) \approx 10$

5. (2 puntos) En la Semana 3, la búsqueda binaria con 16 animales tardó 4 preguntas. ¿Cuántas preguntas necesitaría una búsqueda binaria con 1.024 animales?

Respuesta: _____

Pauta de corrección

Pregunta	Respuesta esperada	Puntos
1	Finitud, Definición, Entrada, Salida, Efectividad — con explicación coherente de cada una	4 pts (0.8 por propiedad bien explicada)
2	Pseudocódigo que incluya: LEER la lista, un bucle PARA CADA cuadrante, un condicional SI cobertura > 60%, agregar a la lista de seleccionados, ESCRIBIR resultado	6 pts (2 por estructura de bucle correcta, 2 por condicional correcto, 2 por entrada/salida explícitas)
3	Diagrama con: inicio, lectura de temperatura, dos decisiones en rombo (< 5°C, luego 15°C), tres salidas en rectángulo, flechas correctas, fin	4 pts (1 por símbolos correctos, 1 por flujo lógico, 1 por condiciones correctas, 1 por completitud)
4a	Bubble sort: $\sim 1.000.000 / 2 = \sim 500.000$ comparaciones ($n(n-1)/2$)	2 pts (1 si orden de magnitud correcto)
4b	Merge sort: $\sim 1.000 \times 10 = \sim 10.000$ comparaciones	2 pts (1 si orden de magnitud correcto)
5	$\log(1024) = 10$ preguntas	2 pts

Notas pedagógicas

El laboratorio en auditorio con bancas fijas

El espacio frente a la pizarra es la “pista de ordenamiento”. Necesitan ~ 7 metros lineales para 8 personas separadas a 80 cm. Si el frente del auditorio no alcanza, usar el pasillo central.

Alternativa si el espacio es muy reducido: hacer el ordenamiento “sentados” en una fila de bancas. Cada estudiante sostiene su tarjeta en alto. En vez de caminar y cruzarse, los que deben intercambiar **pasan sus tarjetas** por encima del vecino. Menos dramático físicamente pero funciona igual pedagógicamente.

Gestión de 30 alumnos: con 3 equipos de 10 (8 participantes + 2 registradores), las fases 1 y 2 son demostraciones con voluntarios del curso completo. La fase 3 (competencia) es donde los tres equipos compiten simultáneamente — necesitan 3 “pistas” marcadas o turnarse (uno compite mientras los otros observan y cronometran).

Recomendación práctica: marcar solo 1 pista. Los equipos compiten **por turnos** (3 minutos cada uno). Los otros dos equipos observan, cuentan comparaciones, y verifican. Esto también evita el caos de 24 personas moviéndose simultáneamente en un auditorio.

Errores conceptuales esperados

- **“El algoritmo más corto de escribir es el mejor”:** No necesariamente. Burbuja es simple de escribir pero costoso de ejecutar. La simplicidad del código y la eficiencia del algoritmo son cosas distintas.
- **“ $O(n^2)$ significa que tarda n^2 segundos”:** No. $O(n^2)$ significa que el número de operaciones *crece como* n^2 . El tiempo real depende de la velocidad del computador. Pero la *forma* de crecimiento es intrínseca al algoritmo.
- **Confundir “difícil de programar” con “difícil de computar”:** Un algoritmo puede ser sencillo de entender pero computacionalmente costoso (burbuja) o difícil de programar pero eficiente (merge sort).

Conexión con el resto del curso

Concepto de esta semana	Dónde se profundiza
Pseudocódigo	Semana 8 (Python: la sintaxis real)
Condicionales en diagramas de flujo	Semana 10 (if/elif/else en Python)
Bucles (PARA, MIENTRAS)	Semana 11 (for, while en Python)
Algoritmos de ordenamiento	Semana 9 (list.sort()) en Python — ¿qué algoritmo usa?)
Complejidad $O(n)$ vs $O(n^2)$	Semana 11 (bucles anidados y eficiencia)
Búsqueda binaria	Semana 12 (implementar como función en Python)